
Programming Exercise: The Sequential Ephemeral Key (ElGamal Attack)

1. Scenario

You have intercepted communications from a suspect using a custom implementation of the ElGamal Digital Signature Scheme. Upon reverse-engineering the signing software, you discovered a critical flaw in the random number generator (RNG).

Instead of generating a cryptographically secure random ephemeral key (k) for each signature, the software simply increments the previous ephemeral key by 1.

Your Goal: Given two consecutive signatures signed by the same key, recover the Private Key (d).

2. System Parameters & Protocol

The system uses a subgroup variant of ElGamal to ensure valid mathematical properties.

- p : A large prime modulus.
- q : A large prime factor of $(p - 1)$. This is the order of the subgroup.
- g : A generator of the subgroup of order q ($g^q \equiv 1 \pmod{p}$).
- y : The Public Key ($y = g^d \pmod{p}$).
- d : The Private Key (Unknown, $1 < d < q$).

Signing Process

To sign a message hash x , the signer computes:

1. Chooses an ephemeral key k .
2. Calculates $r = g^k \pmod{p}$.
3. Calculates $s = k^{-1} \cdot (x - d \cdot r) \pmod{q}$.

The Vulnerability

For two consecutive signatures (r_1, s_1) and (r_2, s_2) , the ephemeral keys satisfy:

$$k_2 = k_1 + 1$$

3. Input Format

The input is a JSON object containing the public parameters and two signature objects.

```
{
  "p": "integer",
  "q": "integer",
  "g": "integer",
  "y": "integer",
  "sig1": { "x": "integer", "r": "integer", "s": "integer" },
  "sig2": { "x": "integer", "r": "integer", "s": "integer" }
}
```

4. Output Format

Output the decimal value of the Private Key d .

5. Test data

Level 1: Sanity Check (Small Numbers)

Use this dataset to debug your math logic.

```
{
  "p": 467,
  "q": 233,
  "g": 4,
  "y": 308,
  "sig1": {
    "x": 100,
    "r": 159,
    "s": 222
  },
  "sig2": {
    "x": 200,
    "r": 133,
    "s": 14
  }
}
```

Expected Output: 123

Level 2: The Challenge (Big Numbers)

Recover the key from this dataset.

```
{
  "p": 21617692091972978742235166480742862532407156107636789231276846348871806
  "q": 10808846045986489371117583240371431266203578053818394615638423174435903
  "g": 81436515481009478433823507513446080245145084277360800916516346685389842
  "y": 53377414827507974762617387052810151957093712167788816336067341282347621
  "sig1": {
    "x": 7004958796436830548173750808247860662321305213197319868848950120409
    "r": 2144289183543323661748508535543604117528968839992539658714555833075
    "s": 9578212581237917941432747084881128196179354508788807162140588617803
  },
  "sig2": {
    "x": 7948944228808646192883566297506348894200221948529547265350717153384
    "r": 1423701848427498781029751954078122243788758722208730082440242748769
    "s": 1062230788290391829840337029483705419499644605034443688076618199513
  }
}
```

6. Hint

You need to set up a system of linear equations based on the signature generation formula:

$$s \equiv k^{-1}(x - d \cdot r) \pmod{q}$$

This can be rewritten as:

$$s \cdot k + r \cdot d \equiv x \pmod{q}$$

Since you have two equations (from sig1 and sig2) and you know $k_2 = k_1 + 1$, you can eliminate one variable to solve for the other.

7. Deliverables

Please submit a zip file containing:

1. **Source Code:** Your Python script used to solve the challenge.
2. **private.txt:** A text file containing the private key d for the **Level 2** dataset.